

Program 2 : Develop a multistage docker file for container orchestration

Step 1 : mkdir program2
cd program2
nano Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node","dist/index.js"]
```

Step 2 : npm init -y
npm install

```
1rv24mc040-haripriya@haripriya-ASUS:~/Devops_lab/program2$ npm init -y
Wrote to /home/1rv24mc040-haripriya/Devops_lab/program2/package.json:

{
  "name": "program2",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

1rv24mc040-haripriya@haripriya-ASUS:~/Devops_lab/program2$ npm install

up to date, audited 1 package in 115ms

found 0 vulnerabilities
.
```

Step 3 : mkdir src
cd src
nano index.js

```
const express=require("express")
const app=express();
const PORT=3000;
app.get('/',(req,res) => {
    res.send("Hello from Docker");
});
app.listen(PORT,() =>{
    console.log(`Server listening on port ${PORT}`);
});
```

Step 4 : docker build -t program2 .

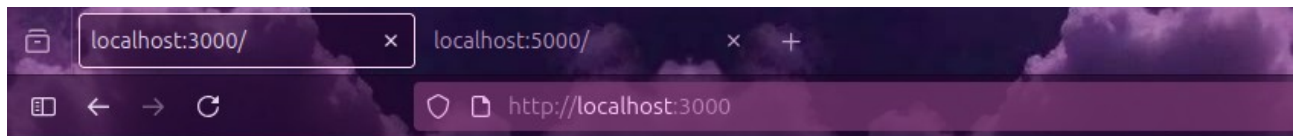
```
1rv24mc040-haripriya@haripriya-ASUS:~/Devops_lab/program2$ docker build -t program2 .
[+] Building 5.8s (15/15) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile               0.0s
=> => transferring dockerfile: 419B                               0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine 1.6s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b97 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 483B                                     0.0s
=> CACHED [builder 2/6] WORKDIR /app                             0.0s
=> [builder 3/6] COPY package.json package-lock.json ./         0.0s
=> [builder 4/6] RUN npm install                                 3.3s
=> [builder 5/6] COPY . .                                         0.0s
=> [builder 6/6] RUN npm run build                               0.4s
=> [stage-1 3/6] COPY --from=builder /app/package.json ./        0.0s
=> [stage-1 4/6] COPY --from=builder /app/package-lock.json ./   0.0s
=> [stage-1 5/6] COPY --from=builder /app/dist ./dist            0.0s
=> [stage-1 6/6] COPY --from=builder /app/node_modules ./node_modules 0.1s
=> exporting to image                                             0.1s
=> => exporting layers                                              0.1s
=> => writing image sha256:15f269c0d733507d65f4d4a7b734970d7f51abc45db29 0.0s
=> => naming image docker.io/library/program2                     0.0s
```

View build details: [docker-desktop://dashboard/build/default/default/xkd2o2hlwyoa0p-ah4wfa2ds45](https://desktop.docker.com/win/en/desktop?version=24.06.0&path=/dashboard/build/default/default/xkd2o2hlwyoa0p-ah4wfa2ds45)

Step 5 : docker run -p 3000:3000 program2

```
1rv24mc040-haripriya@haripriya-ASUS:~/Devops_lab/program2$ docker run -p 3000:3000 program2
Server listening on port 3000
```

Step 6 : Check the output in the browser
<http://localhost:3000>



Hello from Docker